# Python Environment Setup

Practical Assignment • Beginner Guide for Software Testing & Automation

## Objective

Install Python and Visual Studio Code, create a project-specific virtual environment, install the `requests` package, and run a Python script that displays a greeting and the active Python version.

## Task 1 — Install Python and VS Code

### Install Python

1. Visit the official Python Downloads page.
2. Download the latest stable version compatible with Windows and run the installer.
3. If shown, select **Add Python to PATH**, then install Python.
4. Open Command Prompt and verify the installation:

```
python --version
# If needed, try:
py --version
```

### Install Visual Studio Code

1. Visit the official VS Code download page.
2. Install and open VS Code.
3. Open Extensions with **Ctrl + Shift + X**, search for **Python**, and install the Microsoft Python extension.

***Screenshots to capture:*** *(1) Python installation success message or Command Prompt showing the Python version; (2) the VS Code Welcome screen. Use **Win + Shift + S** to take screenshots. These must be captured on your own computer.*

## Task 2 — Create InstaTestProject and set up venv

1. Create a folder named `InstaTestProject` (for example, inside Documents).
2. In VS Code, select **File** → **Open Folder** and open that folder.
3. Open **Terminal** → **New Terminal** and create the virtual environment:

```
python -m venv venv
```

### Activate the environment

For Windows PowerShell:

```
.\venv\Scripts\Activate.ps1
```

For Windows Command Prompt:

```
venv\Scripts\activate.bat
```

*After activation, the terminal prompt should begin with `(venv)`. If PowerShell blocks activation, use Command Prompt or troubleshoot the execution policy carefully.*

## Task 3 — Install requests and list packages

With the virtual environment activated, run:

```
python -m pip install requests
python -m pip list
```

The package list should include `requests` and its dependencies. Exact versions and the complete list can vary.

| Package | Purpose |
|---|---|
| pip | Installs and manages Python packages. |
| requests | Sends HTTP requests; useful for API testing. |
| certifi | Provides a bundle of trusted certificate authorities. |
| charset-normalizer | Helps detect text encodings. |
| idna | Supports internationalized domain names. |
| urllib3 | Provides HTTP client functionality. |

*Capture a screenshot of the actual output from `pip list` in your activated environment.*

## Task 4 — Create and run hello_automation.py

In VS Code Explorer, create a new file named `hello_automation.py` and enter:

```
import sys

print("Hello, Automation Tester!")
print("Python version:", sys.version)
```

Run it from the VS Code terminal:

```
python hello_automation.py
```

### Example output (your version details will be different):

```
Hello, Automation Tester!
Python version: 3.x.x ...
```

**Explanation:** `import sys` imports Python's system module. `sys.version` displays information about the Python interpreter running the script, helping you confirm which Python version the project uses.

***Screenshot to capture:*** *Show the script and its successful output in VS Code.*

## Task 5 — Benefits of a virtual environment (venv)

A Python virtual environment (`venv`) creates an isolated environment for each automation project. It allows different projects to use different package versions without affecting each other. For example, one project can use an older version of `requests`, while another uses a newer version. This prevents dependency conflicts and makes projects easier to manage and maintain.

## Final Submission Checklist

■ Python installation confirmation screenshot

■ VS Code Welcome screen screenshot

■ InstaTestProject folder with venv created and activated

■ requests installed and pip list screenshot

■ hello_automation.py code and successful output screenshot

■ 3–4 line explanation of venv

*Tip: Keep your project files inside InstaTestProject and capture screenshots from your own system for submission.*